📜 รายงานอธิบายโค้ดและหลักการทำงานของระบบ คัมภีร์พูดได้ (Kumpee AI – พระไตรปิฎก AI)

คู่มือเชิงเทคนิคฉบับสมบูรณ์: สถาปัตยกรรมระบบ, โค้ดฝั่ง Client, Local Server, Vercel Serverless และขั้นตอนการ Deploy ขึ้นระบบจริง

> 📌 **สรุปภาพรวมของรายงานฉบับนี้**
>
> เอกสารฉบับนี้รวบรวมคำอธิบายโครงสร้างระบบ หลักการทำงานของโค้ดทุกไฟล์ กลไกการประมวลผลปัญญาประดิษฐ์ (Claude AI), ระบบตรวจสอบพระสูตรตามมหาปเทส 4, ระบบค้นหาข้อมูล (RAG), ระบบแปลงข้อความเป็นเสียงพูด (TTS) รวมถึงขั้นตอนการนำโค้ดขึ้นเผยแพร่บน Vercel Cloud Serverless อย่างเป็นขั้นตอน

1. โครงสร้างไฟล์และสถาปัตยกรรมภาพรวมของระบบ (System Architecture)

ระบบ **คัมภีร์พูดได้ (Kumpee AI)** ถูกออกแบบตามสถาปัตยกรรมแบบ Decoupled Web Application ซึ่งแบ่งภาระหน้าที่อย่างชัดเจนระหว่างฝั่งผู้ใช้ (Client-side Frontend) และฝั่งเซิร์ฟเวอร์ (Server Proxy & Cloud Architecture)

1.1 โครงสร้างโฟลเดอร์และไฟล์ภายในโครงการ

ชื่อไฟล์ / โฟลเดอร์	ประเภท/สภาพแวดล้อม	หน้าที่และคำอธิบาย
`public/app.js`	Front-end (Client)	สคริปต์หลักจัดการ UI, ฐานข้อมูลพระสูตร (RAG), อัลกอริทึมค้นหา, การตรวจสอบรอบ 2 (มหาปเทส 4), เสียงพูด (TTS) และ Speech Recognition
`public/index.html`	Front-end (Client)	โครงสร้าง HTML5 รองรับ Responsive Design หน้าต่างแชต พาเนลคัมภีร์อ้างอิง และตัวอวาตาร์เคลื่อนไหว
`public/index.css`	Front-end (Client)	การจัดสไตล์ visual aesthetics (Theme สีทองโบราณ-เข้ม, Glassmorphism, Micro-animations)
`server.js`	Back-end (Local Server)	Node.js HTTP Server สำหรับรันในเครื่องท้องถิ่น (Port 8080) ทำหน้าที่บริการไฟล์ Static Files และ Proxy ไปยัง Claude API & Google TTS
`api/chat.js`	Serverless (Vercel Cloud)	Vercel Serverless Function (POST /api/chat) สำหรับเชื่อมต่อ Claude API พร้อมระบบยึด System Prompt, ตรวจสอบ Origin, และ Rate Limit
`api/tts.js`	Serverless (Vercel Cloud)	Vercel Serverless Function (GET /api/tts) สำหรับดึงไฟล์เสียงภาษาไทย MP3 จาก Google Translate TTS Stream
`vercel.json`	Config (Vercel)	ไฟล์กำหนดค่า routing, output directory และ CORS headers บน Vercel Cloud Platform
`package.json`	Config (Node.js)	ไฟล์ระบุชื่อโครงการ เวอร์ชัน สคริปต์ build และรุ่นของ Node.js (24.x)
`Dockerfile` & `docker-compose.yml`	Deployment (Container)	ไฟล์สำหรับสั่ง Build และ Deploy แอปพลิเคชันผ่าน Docker Container ในเครื่องหรือบน Server อื่นๆ

2. โครงสร้างและหลักการทำงานฝั่ง Front-end (Client Application)

ไฟล์ `public/app.js` เป็นหัวใจหลักในการรันในเบราว์เซอร์ของผู้ใช้ ถูกออกแบบด้วยรูปแบบ Object Literals แบ่งออกเป็น 6 โมดูลหลัก ดังนี้:

2.1 โมดูล `CONFIG` (ค่าคอนฟิกกลาง)

เก็บค่าคงที่สำหรับปรับแต่งพฤติกรรมของแอปพลิเคชัน เช่น URL ของ Google Sheets CSV (932 รายการ), Endpoint API (`/api/chat`), ชื่อคีย์สำหรับแคช (`tipitaka_data_v3`), ระยะเวลาแคช (24 ชั่วโมง), ความเร็วการพิมพ์ตัวหนังสือ และขีดจำกัดความยาวบริบท

2.2 โมดูล `parseCSV()` และ `DataModule` (การจัดการคลังข้อมูลและการทำ RAG)

- **การโหลดข้อมูล (`DataModule.load`):** ตรวจสอบข้อมูลใน `localStorage` ก่อน หากไม่มีหรือหมดอายุ (เกิน 24 ชม.) จะดึงไฟล์ CSV จาก Google Sheets มาแปลงผ่าน `parseCSV()` ซึ่งรองรับการแกะข้อความที่ซ้อนในเครื่องหมายคำพูด (Quoted fields) ได้อย่างแม่นยำ
- **ระบบแคชอัจฉริยะ (Storage Optimization):** เพื่อป้องกันปัญหาความจุ `localStorage` เต็ม (QuotaExceededError) ระบบจะเลือกบันทึกเฉพาะฟิลด์สำคัญ 11 ฟิลด์ และตัดทอนเนื้อหาให้พอดี (1,000 ตัวอักษรต่อระเบียน) ทำให้ลดขนาดแคชลงได้กว่า 70%
- **ระบบค้นหาขั้นสูง (Weighted Search Ranking):** ฟังก์ชัน `DataModule.search(query)` ใช้อัลกอริทึมให้คะแนนความเกี่ยวข้อง (Scoring System) โดยแบ่งตามความสำคัญ:
  - ตรงกับชื่อพระสูตรแบบเต็ม (+10 คะแนน)
  - ตรงกับเนื้อหาทั้งหมด (+8 คะแนน)
  - คำตรงกับคีย์เวิร์ดคำสอนสำคัญภาษาไทย เช่น *อริยสัจ, ทุกข์, สติ, นิพพาน, ศีล, กรรม* (+3 ถึง +5 คะแนน)
  - ตรงกับส่วนประกอบย่อยของคำถาม (+1 ถึง +4 คะแนน)

2.3 โมดูล `AIModule` (การสืบค้นคำตอบและการตรวจสอบมหาปเทส 4)

- **การส่งคำถาม (Round 1 RAG):** ดึงพระสูตรที่เกี่ยวข้องจาก `DataModule.search()` รวมกันไม่เกิน 4,000 ตัวอักษร แล้วแนบเป็นบริบทส่งไปยัง `/api/chat` พร้อมประวัติการสนทนา 12 ข้อความล่าสุด
- **การตรวจสอบย้อนกลับรอบที่ 2 (Round 2 Verification):** เมื่อได้รับคำตอบจาก AI ระบบจะนำคำตอบที่ได้ไปค้นหาซ้ำในคลังพระไตรปิฎกทันที (`DataModule.search(aiText)`) เพื่อตรวจเช็คว่าเนื้อหาที่ AI ตอบนั้นมีพระสูตรใดในคลังรองรับจริงหรือไม่
- **การประเมินมหาปเทส 4 (`_generateVerdict`):** ระบบจะประมวลผลและสร้างวัตถุโครงสร้าง (Structured JSON Verdict) เพื่อระบุผลการตรวจสอบ 3 รูปแบบ:
  1. สอดคล้อง — คำตอบของ AI ตรงกับพระสูตรที่มีในคลังข้อมูล พร้อมระบุรายชื่อพระสูตรอ้างอิง
  2. ไม่พบหลักฐาน — คำตอบมาจากความรู้ทั่วไปของโมเดล ไม่พบตัวบทพระสูตรยืนยันตรงในคลัง
  3. คำตอบสำรอง (Fallback) — เกิดจากระบบไม่สามารถเชื่อมต่อ AI ได้ จึงดึงข้อมูลพระสูตรอันดับ 1 มาตอบแทน

2.4 โมดูล `SpeechModule` (ระบบเสียงพูด TTS & รับฟังเสียง STT)

- **Google TTS Proxy Stream:** แบ่งข้อความออกเป็นท่อนสั้นๆ (~180 ตัวอักษร) ตามจุดจบประโยค แล้วส่งไปเล่นผ่าน HTML5 `Audio` จาก `/api/tts?text=...` โดยรองรับการปรับระดับความเร็ว (Rate) และโทนเสียง (Tone) เช่น โทนเสียงพระสงฆ์/นุ่มนวล
- **Web Speech API Fallback:** หากเบราว์เซอร์รองรับ สามารถเลือกใช้เอนจิน `SpeechSynthesisUtterance` ภาษาไทยในเครื่องได้
- **Speech Recognition (STT):** ใช้ `webkitSpeechRecognition` ถอดเสียงพูดภาษาไทยเป็นข้อความ แล้วส่งเข้าช่องพิมพ์คำถามให้อัตโนมัติ

2.5 โมดูล `BookModule`, `NavigatorModule` และ `ChatModule` (การแสดงผล UI)

- **BookModule:** แสดงการ์ดผลการตรวจสอบ `มหาปเทส 4` (Verdict Card) และแบ่งส่วนพระสูตรอ้างอิงเป็น 2 กรอบ (กรอบสีเขียว: พระสูตรยืนยันรอบ 2 / กรอบสีทอง: พระสูตรประกอบบริบท)
- **NavigatorModule:** อวาตาร์ 2D ขยับปากตามจังหวะเสียงพูด (Lip-sync Loop) โดยใช้การตัดภาพพื้นหลังขาวออกด้วย HTML5 Canvas (Anti-aliasing edge softening) และรองรับการลากวาง (Draggable) ทั้งบนคอมพิวเตอร์และมือถือ
- **ChatModule & App Controller:** จัดการพิมพ์ตัวหนังสือแบบ Typewriter Animation, ปุ่มหยุดอ่านเสียง (Stop TTS), ชิปคำถามแนะนำ และจัดการส่งคำขอสนทนา

3. โครงสร้างและหลักการทำงานเมื่อรันในเครื่องท้องถิ่น (Local Server – `server.js`)

เมื่อผู้ใช้รันระบบในเครื่องตนเองผ่านคำสั่ง `npm start` หรือ `node server.js` (หรือผ่าน `start.bat` / `start.ps1`) ไฟล์ `server.js` จะทำหน้าที่เป็น Full Node.js Web Server บนพอร์ต `8080`

3.1 การทำงานของ HTTP Server และ Static File Serving

- ใช้ Node.js native module (`http`, `https`, `fs`, `path`) โดยไม่ต้องพึ่งพา External Framework ใหญ่ๆ ทำให้โหลดเร็วและกินทรัพยากรต่ำ
- ฟังก์ชัน `serveStaticFile()` จะทำหน้าที่บริการไฟล์ `index.html`, `index.css`, `app.js` และรูปภาพจากโฟลเดอร์ `public/` พร้อมกำหนด MIME Types (`text/html`, `application/javascript` ฯลฯ) และป้องกันภัยคุกคาม Directory Traversal

3.2 การทำ API Proxy ไปยัง Anthropic Claude API (`handleChatProxy`)

- **Server-Side System Prompt:** กำหนดบทบาท "คัมภีร์พูดได้" (พระไตรปิฎกโบราณที่มีชีวิต) ไว้ในคีย์คงที่ `SYSTEM_PROMPT` บนเซิร์ฟเวอร์เท่านั้น เพื่อป้องกันไม่ให้ผู้ใช้แอบแก้ไขหรือส่ง Prompt แปลกปลอมเข้ามาจากฝั่ง Client
- **Rate Limiting (จำกัดความถี่):** ฟังก์ชัน `isRateLimited(ip)` ตรวจสอบ IP ผู้ยิงคำขอ โดยจำกัดไม่เกิน **15 ครั้งต่อนาทีต่อ IP** หากเกินจะส่ง HTTP Status `429 Too Many Requests`
- **Sanitization (การกรองข้อความ):** จำกัดประวัติสนทนาไม่เกิน 20 ข้อความล่าสุด และจำกัดความยาวไม่เกิน 6,000 ตัวอักษรต่อข้อความ รวมถึงจำกัดขนาด Request Body ไม่เกิน 2MB
- **การเชื่อมต่อ Claude API:** ส่ง Request ด้วยโปรโตคอล `https.request` ไปยัง Endpoint `https.api.anthropic.com/v1/messages` โดยใช้โมเดล `claude-haiku-4-5-20251001`

3.3 ระบบ Google TTS Proxy (`handleTTSProxy`)

```
GET /api/tts?text=ข้อความที่ต้องการอ่าน
```

- รับข้อความเข้ารหัส (URI Encoded) ส่งต่อไปยัง Google Translate TTS Service (`translate.google.com/translate_tts?ie=UTF-8&tl=th&client=tw-ob`)
- รองรับการติดตาม HTTP Redirect (Status 301/302) อัตโนมัติ แล้ว Pipe สตรีมไฟล์เสียง `audio/mpeg` กลับมายังเบราว์เซอร์ พร้อมกำหนด Header `Cache-Control: public, max-age=86400` (แคชเสียงไว้ 1 วัน)

3.4 ระบบ Smart Fallback Engine ในเครื่อง

หากเครื่องไม่ได้ต่ออินเทอร์เน็ต หรือ Claude API ขัดข้อง ฟังก์ชัน `generateSmartFallback()` จะทำงานทันที โดยสกัดข้อมูลพระสูตรอันดับ 1 จากบริบทที่แนบมา นำมาเรียบเรียงเป็นคำตอบคำสอนทางธรรมที่สมบูรณ์ แล้วส่งกลับในรูปแบบ JSON พร้อมติดแท็ก `_fallback: true` ให้ฝั่งหน้าเว็บทราบ

4. โครงสร้างและหลักการทำงานบนเซิร์ฟเวอร์ Cloud (Vercel Serverless Architecture)

เมื่อนำโครงการไปปรับใช้บน Vercel Cloud Platform สถาปัตยกรรมจะเปลี่ยนจาก Server เดี่ยว มาเป็น **Serverless Microservices Architecture** ซึ่งแต่ละ API จะทำงานแยกกันแบบคุ้มค่าและรองรับผู้ใช้จำนวนมากได้พร้อมกัน

4.1 ไฟล์การกำหนดค่า `vercel.json`

```
{ "framework": null, "outputDirectory": "public", "headers": [ { "source": "/api/(.*)", "headers": [ { "key": "Access-Control-Allow-Origin", "value": "*" }, { "key": "Access-Control-Allow-Methods", "value": "GET, POST, OPTIONS" }, { "key": "Access-Control-Allow-Headers", "value": "Content-Type" } ] } ] }
```

ไฟล์นี้บอกให้ Vercel ทราบว่าหน้าเว็บ static อยู่ในโฟลเดอร์ `public` และเปิดให้เรียกใช้งาน API ย่อยผ่านการตั้งค่า CORS Headers

4.2 Vercel Serverless Endpoint 1: `api/chat.js` (POST /api/chat)

ไฟล์นี้จะถูก Vercel เปลี่ยนให้กลายเป็น Cloud Function (AWS Lambda / Edge Node) ที่จะทำงานเฉพาะเมื่อมีคำขอส่งเข้ามาเท่านั้น:

1. **การตรวจสอบ Origin / Referer (Security Check):**

   ระบบจะตรวจสอบ Header `Origin` ของคำขอ หากไม่ได้มาจากโดเมนที่ได้รับอนุญาต (เช่น `kumpee-ai-psi.vercel.app` หรือ `localhost`) ระบบจะปฏิเสธคำขอทันทีด้วย HTTP Status `403 Forbidden` เพื่อป้องกันผู้อื่นนำ API Proxy ไปแอบใช้งาน

2. **ระบบจำกัดความถี่ (In-Memory Rate Limiting):**

   ตรวจสอบ IP ของผู้ใช้งานผ่าน `x-forwarded-for` โดยจำกัดไม่เกิน 15 คำขอต่อนาที หากเกินจะส่ง `429 Too Many Requests`

3. **การล็อก System Prompt ฝั่ง Server:**

   ระบบจะนำข้อความสนทนาจากผู้ใช้มาทำ Sanitization แล้วส่งไปหา Claude API (`claude-haiku-4-5-20251001`) ร่วมกับ `SYSTEM_PROMPT` ที่กำหนดไว้ในคลาวด์เท่านั้น โดยไม่อ่านค่าคีย์หรือ System Prompt จากฝั่งผู้ใช้

4. **Smart Fallback บนคลาวด์:**

   หาก Claude API ส่งผลลัพธ์ผิดพลาด เซิร์ฟเวอร์จะสร้างคำตอบสำรองที่ดึงเนื้อหาพระสูตรที่เกี่ยวข้องมาจัดรูปแบบให้อย่างสวยงาม

4.3 Vercel Serverless Endpoint 2: `api/tts.js` (GET /api/tts)

ทำหน้าที่เป็นสตรีมมิงพร็อกซีแปลงข้อความภาษาไทยเป็นไฟล์เสียง MP3 จาก Google TTS และส่งบัฟเฟอร์เสียง (ArrayBuffer / Buffer) กลับไปที่หน้าเว็บ พร้อมแคชไฟล์เสียงไว้บน CDN ของ Vercel เพื่อความรวดเร็วในการเล่นครั้งถัดไป

5. ขั้นตอนการอัปเดตและ Deploy โค้ดขึ้นเซิร์ฟเวอร์ (Vercel Deployment Process)

การอัปเดตโค้ดจากเครื่องคอมพิวเตอร์ขึ้นสู่ Vercel Production Server ทำได้อย่างรวดเร็วและปลอดภัย ผ่าน Vercel Command Line Interface (Vercel CLI)

ขั้นตอนที่ 1: การเตรียมโค้ดและตรวจสอบความพร้อมในเครื่อง

- ตรวจสอบไฟล์โค้ดในโฟลเดอร์โครงการ `d:\project\ไตรปิฎก` ว่าไฟล์ `public/app.js`, `api/chat.js`, `api/tts.js` และ `server.js` ได้รับการบันทึกการแก้ไขเรียบร้อยแล้ว
- ตรวจสอบว่าการตั้งค่าโมเดลเป็น `claude-haiku-4-5-20251001` ทั้งหมด

ขั้นตอนที่ 2: การตั้งค่า Environment Variables บน Vercel

หากต้องการระบุ API Key ผ่านตัวแปรสภาพแวดล้อม สามารถสั่งงานผ่าน Vercel CLI ได้ดังนี้:

```
npx vercel env add CLAUDE_API_KEY production npx vercel env add CLAUDE_MODEL production
```

ขั้นตอนที่ 3: คำสั่งสั่งอัปโหลดและ Deploy สู่ Production

เปิด PowerShell ในโฟลเดอร์โครงการ แล้วรันคำสั่งเพียงบรรทัดเดียว:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass; npx -y vercel --prod --project kumpee-ai --scope stl-s --yes
```

คำอธิบายพารามิเตอร์ของคำสั่ง:

- `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` — อนุญาตให้รันสคริปต์ Node/npx ในเซสชัน PowerShell ปัจจุบัน
- `npx -y vercel` — เรียกใช้งาน Vercel CLI เวอร์ชันล่าสุดโดยอัตโนมัติ
- `--prod` — ระบุเป้าหมายให้อัปเดตเข้าสู่สภาพแวดล้อม Production (ไม่ใช่แค่ Preview)
- `--project kumpee-ai` — ระบุชื่อโครงการบน Vercel ให้ตรงกัน
- `--scope stl-s` — ระบุทีม/บัญชีผู้ดูแลโครงการ
- `--yes` — ยืนยันการอัปโหลดอัตโนมัติโดยไม่ต้องรอให้ผู้ใช้กดตอบ Y/N

ขั้นตอนที่ 4: สิ่งที่ Vercel ทำเบื้องหลังขณะ Deploy (Build & Routing)

1. **Uploading:** Vercel CLI จะบีบอัดไฟล์ในโครงการและอัปโหลดขึ้นสู่คลาวด์
2. **Building:** Vercel จะรันคำสั่ง `npm run build` ตรวจสอบโครงสร้างไฟล์ static และรวบรวมไฟล์ในโฟลเดอร์ `api/` แยกสร้างเป็น Serverless Functions (`api/chat` และ `api/tts`) ที่ Washington D.C. Data Center (iad1)
3. **Completing & Aliasing:** เมื่อการสร้างระบบเสร็จสมบูรณ์ Vercel จะสร้าง URL การอัปเดตย่อย (เช่น `kumpee-ec8cngqj0-stl-s.vercel.app`) แล้วสลับโดเมนหลัก (Domain Alias) ทั้งหมด ได้แก่ `https://kumpee-ai-psi.vercel.app` ให้ชี้มายังโค้ดเวอร์ชันล่าสุดแบบ Zero-Downtime (ผู้ใช้งานไม่รู้สึกว่าเว็บกระตุกหรือดับไปขณะอัปเดต)

ขั้นตอนที่ 5: การตรวจสอบผลลัพธ์หลังการ Deploy (Verification)

หลังการ Deploy สามารถทดสอบความถูกต้องของระบบบนเซิร์ฟเวอร์จริงได้ผ่าน 3 วิธี:

- **การเช็คความตรงกันของไฟล์ Static:** ตรวจสอบค่า MD5 Hash ของไฟล์ `app.js` , `index.html` , `index.css` ระหว่างเครื่องและเซิร์ฟเวอร์ให้ตรงกัน 100%
- **การทดสอบ API Endpoint:** สั่งยิงคำขอทดสอบไปที่ `POST /api/chat` เพื่อยืนยันว่าตอบกลับด้วยโมเดล `claude-haiku-4-5-20251001` และตอบกลับเป็นภาษาไทย
- **การดูไดอารี่บันทึกระบบ (Vercel Logs):** เรียกดูบันทึกเหตุการณ์จริงผ่านคำสั่ง `npx vercel logs https://kumpee-ai-psi.vercel.app` เพื่อตรวจสอบว่าไม่มีข้อผิดพลาด (Error Log) เกิดขึ้น

6. สรุปขั้นตอนการทำงานทั้งหมดตั้งแต่ต้นจนจบ (End-to-End Execution Flow)

เมื่อผู้ใช้งานเปิดหน้าเว็บขึ้นมา การทำงานตั้งแต่ต้นจนจบจะดำเนินไปตามลำดับขั้นตอนดังนี้:

🔄 ลำดับขั้นตอนการทำงานของระบบเมื่อผู้ใช้พิมพ์คำถาม (Sequence Diagram Flow)

1. **ผู้ใช้พิมพ์คำถาม:** เช่น "อริยสัจ 4 คืออะไร" ลงในช่องแชต หรือกดพูดผ่านไมโครโฟน (STT)
2. **ค้นหาคลังข้อมูล (Round 1 RAG):** `DataModule.search()` ค้นหาพระสูตรที่เกี่ยวข้องที่สุดจาก 932 รายการในคลังข้อมูล (ที่โหลดแคชมาจาก Google Sheets)
3. **ส่งคำขอไปยัง Serverless Proxy:** `AIModule.chat()` แนบข้อความพระสูตรประกอบบริบท ยิง Request ไปยัง `POST /api/chat` บน Vercel Serverless Server
4. **ประมวลผลบน Cloud Proxy:** `api/chat.js` ตรวจสอบ Origin, เช็ค Rate Limit, ผนวก `SYSTEM_PROMPT` ฝั่งเซิร์ฟเวอร์ แล้วยิงคำขอไปยัง Anthropic Claude API (`claude-haiku-4-5-20251001`)
5. **ตอบกลับสู่ Client:** Claude API คืนคำตอบทางธรรมภาษาไทยส่งกลับมายังหน้าเว็บ
6. **ตรวจสอบรอบที่ 2 (มหาปเทส 4):** หน้าเว็บนำคำตอบ AI ที่ได้ ไปค้นหาซ้ำในคลังพระสูตร (`DataModule.search(aiText)`) แล้วคำนวณสร้างการ์ดผลการตรวจสอบมหาปเทส 4 (Verdict Card: สอดคล้อง / ไม่พบหลักฐาน)
7. **แสดงผลบน UI & อ้างอิง:** `ChatModule` พิมพ์ข้อความคำตอบแบบ Typewriter Animation, `BookModule` แสดงพาเนลอ้างอิงพระสูตรยืนยัน (กรอบเขียว) และพระสูตรบริบท (กรอบทอง)
8. **อ่านออกเสียง (TTS):** `SpeechModule` ส่งข้อความคำตอบย่อยไปที่ `GET /api/tts` เพื่อดึงไฟล์เสียง MP3 มาอ่านออกเสียงภาษาไทย พร้อมสั่งให้อวาตาร์ `NavigatorModule` ขยับปากแบบ Lip-sync เสมือนมีชีวิตจริง